Date.prototype.setDate()

📄 Título

Date.prototype.setDate()

📝 Definición

`Date.prototype.setDate()` es un método de JavaScript que establece el **día del mes** (1-31) de un objeto `Date` según la **hora local**, ajustando automáticamente el mes y el año si es necesario.

🧠 ¿Qué es?

Es el método estándar para **modificar el día** de una fecha en JavaScript.

A diferencia de otros métodos que se enfocan en el año o mes, `setDate()` permite cambiar el **día del mes** manteniendo el mes y año existentes, con la capacidad de manejar **overflows** (ej: pasar del 31 de enero al 1 de febrero).

El método es **mutable**: modifica el objeto `Date` original en lugar de crear uno nuevo.

🎯 ¿Para qué sirve?

- Cambiar el **día** de una fecha existente.
- Sumar o restar **días** a una fecha.
- Implementar navegación de **calendarios** (día siguiente/anterior).
- Establecer fechas **específicas** en un mes.
- Manejar **ajustes automáticos** de fin de mes.
- Generar **fechas recurrentes** (ej: mismo día cada mes).
- Implementar **cálculos de fechas** basados en días.
- Crear **rangos de fechas** día a día.

⚙️ ¿Cómo funciona?

El método establece el día del mes (1-31) basado en la **hora local**.

Comportamiento clave:

- Si el día está dentro del rango del mes actual, se establece directamente.
- Si el día es mayor que los días del mes, el método **ajusta automáticamente** al mes siguiente.
  - Ej: `setDate(32)` en enero → 1 de febrero.
- Si el día es 0 o negativo, se ajusta al mes anterior.
  - Ej: `setDate(0)` en enero → 31 de diciembre del año anterior.
  - Ej: `setDate(-1)` → 30 de diciembre del año anterior.
- La hora, minutos, segundos y milisegundos se mantienen sin cambios.

Retorna: El **timestamp** en milisegundos del objeto modificado.

📝 Sintaxis / Estructura (JavaScript)

```
date.setDate(day)
```

Parámetros:

- `day` (obligatorio): Número entero que representa el día del mes (1-31, aunque se aceptan valores fuera de rango con ajuste automático).

Retorna:

- El **timestamp** en milisegundos (del objeto modificado).

Errores:

- Si el parámetro es inválido (ej: `NaN`), el objeto `Date` se vuelve inválido.

💡 Ejemplos

```
// 1. Uso básico - establecer día específico
const date = new Date(2024, 11, 25);
```

```
console.log(date.getDate()); // 25

date.setDate(15);
console.log(date.getDate()); // 15
console.log(date.getMonth()); // 11 (diciembre)
console.log(date.getFullYear()); // 2024

// 2. Ajuste automático - día mayor que los días del mes
const date2 = new Date(2024, 0, 31); // 31 de enero
date2.setDate(32);
console.log(date2.getDate()); // 1
console.log(date2.getMonth()); // 1 (febrero)
console.log(date2.getFullYear()); // 2024

// 3. Ajuste automático - día 0 (último día del mes anterio
r)
const date3 = new Date(2024, 0, 15); // 15 de enero
date3.setDate(0);
console.log(date3.getDate()); // 31
console.log(date3.getMonth()); // 11 (diciembre del año ant
erior)
console.log(date3.getFullYear()); // 2023

// 4. Ajuste automático - día negativo
const date4 = new Date(2024, 0, 15);
date4.setDate(-1);
console.log(date4.getDate()); // 30
console.log(date4.getMonth()); // 11 (diciembre 2023)

// 5. Sumar días a una fecha
function addDays(date, days) {
  const newDate = new Date(date);
  newDate.setDate(newDate.getDate() + days);
  return newDate;
}

const start = new Date(2024, 0, 1);
console.log(addDays(start, 5)); // 2024-01-06
```

```
console.log(addDays(start, 31)); // 2024-02-01
console.log(addDays(start, -1)); // 2023-12-31

// 6. Último día del mes
function getLastDayOfMonth(year, month) {
  const date = new Date(year, month + 1, 0);
  return date.getDate();
}

console.log(getLastDayOfMonth(2024, 0)); // 31 (enero)
console.log(getLastDayOfMonth(2024, 1)); // 29 (febrero bis
iesto)
console.log(getLastDayOfMonth(2023, 1)); // 28 (febrero no
bisiesto)

// 7. Establecer el primer día del mes
function setFirstDayOfMonth(date) {
  const newDate = new Date(date);
  newDate.setDate(1);
  return newDate;
}

const d = new Date(2024, 6, 15);
console.log(setFirstDayOfMonth(d)); // 2024-07-01

// 8. Establecer el último día del mes
function setLastDayOfMonth(date) {
  const newDate = new Date(date);
  const year = newDate.getFullYear();
  const month = newDate.getMonth();
  const lastDay = getLastDayOfMonth(year, month);
  newDate.setDate(lastDay);
  return newDate;
}

const d2 = new Date(2024, 6, 15);
console.log(setLastDayOfMonth(d2)); // 2024-07-31
```

```javascript
// 9. Navegación de calendario (día siguiente/anterior)
function nextDay(date) {
  const newDate = new Date(date);
  newDate.setDate(newDate.getDate() + 1);
  return newDate;
}

function prevDay(date) {
  const newDate = new Date(date);
  newDate.setDate(newDate.getDate() - 1);
  return newDate;
}

const today = new Date(2024, 6, 15);
console.log(nextDay(today)); // 2024-07-16
console.log(prevDay(today)); // 2024-07-14

// 10. Días entre fechas (usando setDate)
function daysBetween(date1, date2) {
  const d1 = new Date(date1);
  const d2 = new Date(date2);

  // Resetear horas para comparar solo días
  d1.setHours(0, 0, 0, 0);
  d2.setHours(0, 0, 0, 0);

  const diff = Math.abs(d2.getTime() - d1.getTime());
  return Math.floor(diff / (1000 * 60 * 60 * 24));
}

console.log(daysBetween(
  new Date(2024, 0, 1),
  new Date(2024, 0, 31)
)); // 30

// 11. Sumar meses manteniendo el día (con ajuste)
function addMonths(date, months) {
  const newDate = new Date(date);
```

```
  const currentDay = newDate.getDate();

  newDate.setMonth(newDate.getMonth() + months);

  // Si el día original no existe en el mes nuevo, ajustar
al último día
  if (newDate.getDate() !== currentDay) {
    newDate.setDate(0);
  }

  return newDate;
}

console.log(addMonths(new Date(2024, 0, 31), 1)); // 2024-0
2-29 (bisiesto)
console.log(addMonths(new Date(2023, 0, 31), 1)); // 2023-0
2-28

// 12. Generar rango de fechas
function dateRange(startDate, endDate) {
  const dates = [];
  const current = new Date(startDate);

  while (current <= endDate) {
    dates.push(new Date(current));
    current.setDate(current.getDate() + 1);
  }

  return dates;
}

const range = dateRange(
  new Date(2024, 6, 1),
  new Date(2024, 6, 10)
);
console.log(range.length); // 10
console.log(range[0].getDate()); // 1
```

```
// 13. Día de la semana para una fecha específica
function getDayOfWeek(year, month, day) {
  const date = new Date(year, month, day);
  const days = ['domingo', 'lunes', 'martes', 'miércoles',
'jueves', 'viernes', 'sábado'];
  return days[date.getDay()];
}

console.log(getDayOfWeek(2024, 11, 25)); // "miércoles"

// 14. Ajustar día manteniendo mes y año
function setDayOfMonth(date, day) {
  const newDate = new Date(date);
  newDate.setDate(day);
  return newDate;
}

const d3 = new Date(2024, 6, 20);
console.log(setDayOfMonth(d3, 5)); // 2024-07-05

// 15. Validar si una fecha existe
function isValidDate(year, month, day) {
  const date = new Date(year, month, day);
  return date.getFullYear() === year &&
         date.getMonth() === month &&
         date.getDate() === day;
}

console.log(isValidDate(2024, 1, 29)); // true (bisiesto)
console.log(isValidDate(2023, 1, 29)); // false (no bisiest
o)
console.log(isValidDate(2024, 11, 31)); // true

// 16. Cadena de métodos (retorna timestamp)
const date5 = new Date();
const timestamp = date5.setDate(15);
console.log(timestamp); // Número (timestamp en ms)
console.log(date5.getDate()); // 15
```

⚠️ Errores comunes

- **No considerar el ajuste automático**:

```
const date = new Date(2024, 0, 31); // 31 de enero
date.setDate(32);
console.log(date.getDate()); // 1 (febrero) - ¡No es 32!
```

- **Pasar valores fuera de rango sin entender el comportamiento**:

```
// ❌ Asumir que setDate(0) daría error
date.setDate(0); // ✅ Válido: último día del mes anteri
or

// ❌ Asumir que setDate(-5) daría error
date.setDate(-5); // ✅ Válido: ajuste hacia atrás
```

- **Confundir `setDate()` con `setDay()`**:

```
// setDate() establece el DÍA DEL MES (1-31)
// ¡NO existe setDay() para establecer el día de la sema
na!
```

- **No considerar meses bisiestos**:

```
const date = new Date(2023, 1, 28); // 28 de febrero 202
3
date.setDate(29);
console.log(date.getDate()); // 1 (marzo)
// No es válido para años no bisiestos
```

- **Olvidar que setDate() modifica el objeto original**:

```
const original = new Date(2024, 6, 15);
const copy = original;
copy.setDate(20);
console.log(original.getDate()); // 20 (¡también modific
ado!)

// Para evitar: const copy = new Date(original);
```

💼 Casos de uso

1. **Sistema de recordatorios con fechas recurrentes**:

```
class ReminderManager {
  constructor() {
    this.reminders = [];
  }

  addMonthlyReminder(name, day, startDate = new Date())
{
    const start = new Date(startDate);
    const currentYear = start.getFullYear();
    const currentMonth = start.getMonth();

    // Crear fecha para el día especificado en el mes ac
tual
    const date = new Date(currentYear, currentMonth, da
y);

    this.reminders.push({
      name,
      day,
      nextDate: date,
      active: true
    });
```

```
    return this.reminders[this.reminders.length - 1];
  }

  getNextOccurrence(reminder) {
    const now = new Date();
    const currentYear = now.getFullYear();
    const currentMonth = now.getMonth();

    let date = new Date(currentYear, currentMonth, remin
der.day);

    // Si ya pasó este mes, pasar al siguiente
    if (date < now) {
      date.setMonth(date.getMonth() + 1);
      // Ajustar el día por si el mes siguiente tiene me
nos días
      date.setDate(reminder.day);
    }

    return date;
  }

  getUpcomingReminders() {
    const now = new Date();
    const future = new Date(now);
    future.setDate(future.getDate() + 7);

    return this.reminders
      .filter(r => r.active)
      .map(r => ({
        ...r,
        nextDate: this.getNextOccurrence(r)
      }))
      .filter(r => r.nextDate >= now && r.nextDate <= fu
ture)
      .sort((a, b) => a.nextDate - b.nextDate);
  }
```

```
  markCompleted(name) {
    const reminder = this.reminders.find(r => r.name ===
name);
    if (!reminder) return null;

    const nextDate = this.getNextOccurrence(reminder);
    reminder.nextDate = nextDate;

    return reminder;
  }
}

const reminders = new ReminderManager();
reminders.addMonthlyReminder('Pagar alquiler', 1);
reminders.addMonthlyReminder('Revisión de proyecto', 1
5);
console.log(reminders.getUpcomingReminders());
```

2. **Calendario de navegación mensual**:

```
class CalendarNavigator {
  constructor(year = new Date().getFullYear(), month = n
ew Date().getMonth()) {
    this.year = year;
    this.month = month;
  }

  getDaysInMonth() {
    const date = new Date(this.year, this.month + 1, 0);
    return date.getDate();
  }

  getFirstDayOfMonth() {
    const date = new Date(this.year, this.month, 1);
    return date.getDay();
  }
```

```
  getMonthGrid() {
    const daysInMonth = this.getDaysInMonth();
    const firstDay = this.getFirstDayOfMonth();
    const grid = [];

    // Días del mes anterior
    const prevMonthDays = new Date(this.year, this.mont
h, 0).getDate();
    for (let i = firstDay - 1; i >= 0; i--) {
      const date = new Date(this.year, this.month, 0);
      date.setDate(prevMonthDays - i);
      grid.push({
        date,
        day: date.getDate(),
        isCurrentMonth: false
      });
    }

    // Días del mes actual
    for (let day = 1; day <= daysInMonth; day++) {
      const date = new Date(this.year, this.month, day);
      grid.push({
        date,
        day,
        isCurrentMonth: true
      });
    }

    // Días del mes siguiente
    const totalDays = grid.length;
    const remainingDays = 7 - (totalDays % 7);
    if (remainingDays < 7) {
      for (let day = 1; day <= remainingDays; day++) {
        const date = new Date(this.year, this.month + 1,
day);
        grid.push({
          date,
          day: date.getDate(),
```

```
          isCurrentMonth: false
        });
      }
    }

    return grid;
  }

  nextMonth() {
    this.month++;
    if (this.month > 11) {
      this.month = 0;
      this.year++;
    }
  }

  prevMonth() {
    this.month--;
    if (this.month < 0) {
      this.month = 11;
      this.year--;
    }
  }

  goToToday() {
    const today = new Date();
    this.year = today.getFullYear();
    this.month = today.getMonth();
  }
}

const calendar = new CalendarNavigator(2024, 6);
console.log(calendar.getMonthGrid());
calendar.nextMonth();
console.log(calendar.month); // 7 (agosto)
```

3. **Sistema de fechas límite y plazos**:

```
class DeadlineManager {
  constructor() {
    this.tasks = [];
  }

  addTask(name, deadlineDate, reminderDays = 3) {
    const deadline = new Date(deadlineDate);
    const reminder = new Date(deadline);
    reminder.setDate(reminder.getDate() - reminderDays);

    this.tasks.push({
      name,
      deadline,
      reminder,
      reminderDays,
      status: 'pending'
    });

    return this.tasks[this.tasks.length - 1];
  }

  extendDeadline(name, extraDays) {
    const task = this.tasks.find(t => t.name === name);
    if (!task) return null;

    task.deadline.setDate(task.deadline.getDate() + extr
aDays);
    task.reminder.setDate(task.reminder.getDate() + extr
aDays);

    return task;
  }

  getOverdue() {
    const now = new Date();
    return this.tasks
      .filter(t => t.status === 'pending' && t.deadline
< now)
```

```
      .sort((a, b) => a.deadline - b.deadline);
  }

  getUpcomingDeadlines(daysThreshold = 7) {
    const now = new Date();
    const future = new Date(now);
    future.setDate(future.getDate() + daysThreshold);

    return this.tasks
      .filter(t => t.status === 'pending' && t.deadline
>= now && t.deadline <= future)
      .sort((a, b) => a.deadline - b.deadline);
  }

  getTasksByDay(date) {
    const target = new Date(date);
    target.setHours(0, 0, 0, 0);

    return this.tasks.filter(t => {
      const deadline = new Date(t.deadline);
      deadline.setHours(0, 0, 0, 0);
      return deadline.getTime() === target.getTime();
    });
  }
}

const deadlines = new DeadlineManager();
deadlines.addTask('Proyecto A', new Date(2024, 6, 15),
5);
deadlines.addTask('Proyecto B', new Date(2024, 6, 20),
3);
console.log(deadlines.getUpcomingDeadlines());
```

4. **Generador de fechas de pago**:

```
class PayrollGenerator {
  constructor() {
```

```
    this.payrolls = [];
  }

  generatePayroll(year, month, payDay) {
    // Verificar que el día existe en el mes
    const lastDay = new Date(year, month + 1, 0).getDate
();
    const actualPayDay = Math.min(payDay, lastDay);

    const payDate = new Date(year, month, actualPayDay);

    this.payrolls.push({
      year,
      month,
      payDay: actualPayDay,
      payDate,
      status: 'pending'
    });

    return this.payrolls[this.payrolls.length - 1];
  }

  generateAnnualPayrolls(year, payDay) {
    const payrolls = [];
    for (let month = 0; month < 12; month++) {
      payrolls.push(this.generatePayroll(year, month, pa
yDay));
    }
    return payrolls;
  }

  getUpcomingPayments(daysThreshold = 30) {
    const now = new Date();
    const future = new Date(now);
    future.setDate(future.getDate() + daysThreshold);

    return this.payrolls
      .filter(p => p.status === 'pending')
```

```
      .filter(p => p.payDate >= now && p.payDate <= futu
re)
      .sort((a, b) => a.payDate - b.payDate);
  }

  markPaid(year, month) {
    const payroll = this.payrolls.find(
      p => p.year === year && p.month === month
    );
    if (!payroll) return null;

    payroll.status = 'paid';
    return payroll;
  }
}

const payroll = new PayrollGenerator();
payroll.generateAnnualPayrolls(2024, 15);
console.log(payroll.getUpcomingPayments(60));
```

5. **Sistema de turnos rotativos**:

```
class ShiftManager {
  constructor() {
    this.shifts = [];
  }

  createShift(name, startDate, frequency, shiftLength) {
    const start = new Date(startDate);
    const end = new Date(start);
    end.setDate(end.getDate() + shiftLength - 1);

    this.shifts.push({
      name,
      start,
      end,
      frequency, // 'daily', 'weekly', 'biweekly'
```

```
      shiftLength,
      current: true,
      history: []
    });

    return this.shifts[this.shifts.length - 1];
  }

  rotateShifts() {
    this.shifts.forEach(shift => {
      if (!shift.current) return;

      const newStart = new Date(shift.end);
      newStart.setDate(newStart.getDate() + 1);

      const newEnd = new Date(newStart);
      newEnd.setDate(newEnd.getDate() + shift.shiftLengt
h - 1);

      shift.history.push({
        start: shift.start,
        end: shift.end
      });

      shift.start = newStart;
      shift.end = newEnd;
    });
  }

  getCurrentShifts() {
    const now = new Date();
    return this.shifts
      .filter(s => s.current)
      .filter(s => s.start <= now && s.end >= now);
  }

  getUpcomingShifts(daysThreshold = 7) {
    const now = new Date();
```

```
    const future = new Date(now);
    future.setDate(future.getDate() + daysThreshold);

    return this.shifts
      .filter(s => s.current)
      .filter(s => s.start >= now && s.start <= future)
      .sort((a, b) => a.start - b.start);
  }
}

const shifts = new ShiftManager();
shifts.createShift('Turno A', new Date(2024, 6, 1), 'wee
kly', 7);
shifts.createShift('Turno B', new Date(2024, 6, 8), 'wee
kly', 7);
console.log(shifts.getCurrentShifts());
shifts.rotateShifts();
console.log(shifts.getUpcomingShifts(14));
```

🧩 Relación con otros conceptos

- `Date.prototype.getDate()`: Obtiene el día del mes.
- `Date.prototype.setMonth()`: Establece el mes.
- `Date.prototype.setFullYear()`: Establece el año.
- `Date.prototype.getDay()`: Obtiene el día de la semana.
- `Date.prototype.setHours()`: Establece la hora.
- `Date.prototype.setMinutes()`: Establece los minutos.

🧠 Analogía

Imagina que `setDate()` es como un **calendario de pared ajustable**. Puedes girar una rueda para cambiar el número del día, y si pasas del 31, automáticamente el calendario cambia al mes siguiente. Es una forma simple de navegar por las fechas, manteniendo todo sincronizado.

📚 Fuente

- MDN Web Docs – Date.prototype.setDate()
- ECMAScript Specification – Date.prototype.setDate